



{ یک یادگیری کارساز }

بوتکمپ

Artificial Intelligence

MLOps

Session 5

Session Overview

Model Development & Testing

Learning Objectives:

- Code organization for ML projects
- Configuration-driven development
- Testing strategies for ML systems
- Model validation frameworks
- Bias detection and fairness

Recap - Previous Session

What We Covered:

- Data management and versioning
- Experiment tracking with Mlflow
- VisionaryAI's data pipeline challenges

Today's Focus:

- From data to production-ready code
- Testing and validation strategies

ML Code Challenges

Why ML Code is Different:

- Multiple components: data, models, evaluation
- Experimentation-heavy workflow
- Environment dependency issues
- Iterative development cycles
- Reproducibility requirements

VisionaryAI Example:

- Computer vision defect detection
- Initial notebook chaos → Structured approach

Bad vs Good Code Organization

Poor Structure:

```
defect_detection/  
├── notebook.ipynb  # Everything mixed  
├── data.csv  
└── model.pkl
```

Better Structure:

```
defect_detection/  
├── src/  
│   ├── data/  
│   ├── models/  
│   ├── evaluation/  
│   └── config/  
├── tests/  
├── requirements.txt  
└── README.md
```

Code Organization Principles

Core Principles:

1. Separation of Concerns

- Data processing $\neq$ Model training $\neq$ Evaluation

2. Reusability

- Modular, composable components

3. Testability

- Each component can be tested independently

4. Configurable

- Parameters externalized from code

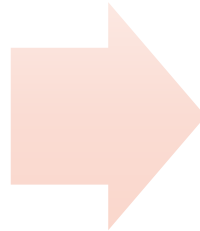
5. Version-controlled

- Everything tracked, not just notebooks

VisionaryAI Case Study - NLP Agent

The Problem:

- Single 2000-line script
- Hardcoded parameters
- Impossible to experiment
- Team collaboration nightmare



The Solution:

- Modular architecture
- Clear interfaces
- Configuration-driven approach
- A/B testing capability

Configuration-Driven Development

Why Configuration Matters:

- Enable experimentation without code changes
- Ensure reproducibility
- Support different environments
- Enable automated pipelines
- Facilitate team collaboration

VisionaryAI Experimenting:

- Embedding dimensions: 128, 256, 512
- Learning rates: 0.001, 0.01, 0.1
- Model architectures: ResNet, EfficientNet
- Preprocessing variations

Configuration File Example

YAML Configuration Structure:

```
yaml
model:
  architecture: "resnet50"
  input_size: [224, 224, 3]
  dropout_rate: 0.2

training:
  batch_size: 32
  learning_rate: 0.001
  epochs: 100

data:
  train_path: "data/train"
  augmentation:
    rotation_range: 20
    horizontal_flip: true
```

Configuration Benefits

Key Benefits:

1. Experiment Tracking
 - Config saved with results
2. Reproducibility
 - Exact recreation possible
3. Environment Management
 - Dev/staging/production configs
4. Collaboration
 - Easy sharing and understanding
5. Automation
 - CI/CD pipeline integration

VisionaryAI's Three Environments:

- Development: Small dataset, fast iteration
- Staging: Production subset, validation
- Production: Full dataset, optimized parameters

Configuration Best Practices

Best Practices:

1. Hierarchical Configs
 - Base + environment overrides
2. Validation
 - Always validate config values
3. Documentation
 - Comment your configurations
4. Version Control
 - Configs alongside code
5. Environment Variables
 - For sensitive information
6. Schema Validation
 - Use Pydantic for structure validation

⚠ **Common Mistake:** Putting secrets in config files

Testing ML Systems - The Challenge

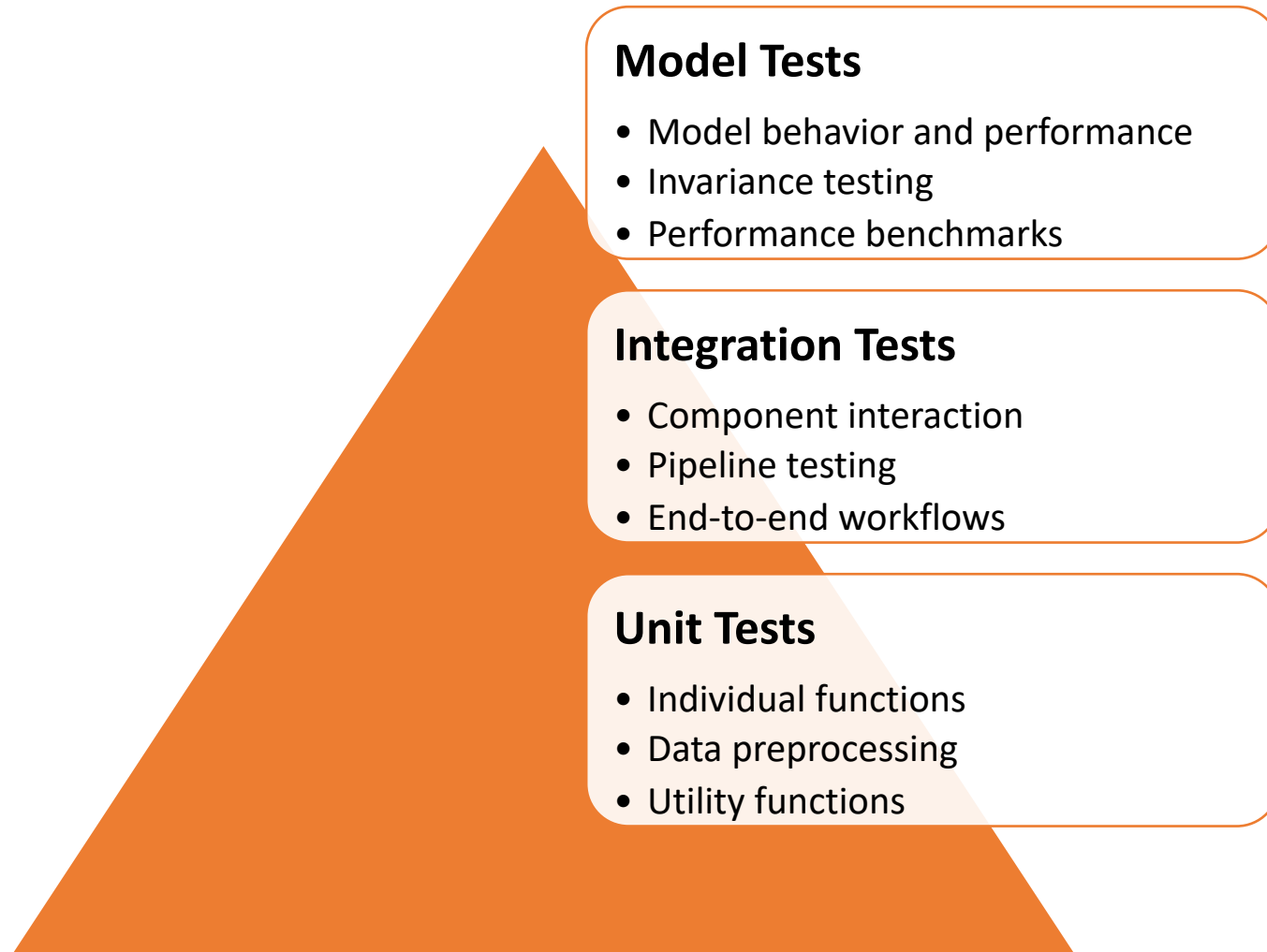
Why ML Testing is Different

- Non-deterministic outputs
- Data-dependent behavior
- Model performance degradation
- Complex data-model interactions

VisionaryAI's Learning

- Unit tests passed 
- Production model failed 
- Issue: Data quality, not code logic

ML Testing Pyramid



Unit Testing

What to Unit Test:

- Data preprocessing functions
- Feature engineering logic
- Configuration parsing
- Utility functions

```
def test_image_preprocessing():  
    input_image = create_test_image()  
    processed = preprocess_image(input_image)  
  
    assert processed.shape == (224, 224, 3)  
    assert processed.dtype == np.float32  
    assert 0 <= processed.min() and  
processed.max() <= 1
```

Integration Testing

Testing Component Interactions:

- Complete pipeline testing
- Data flow validation
- Model training workflows

```
def test_training_pipeline():  
    config = load_test_config()  
    model = create_model(config)  
    train_data = load_test_data()  
  
    trained_model = train_model(model,  
                                train_data, config)  
  
    assert trained_model is not None  
    assert hasattr(trained_model, 'predict')
```


Model-Specific Testing

VisionaryAI's Testing Strategy by Domain:

Computer Vision (Defect Detection):

- Invariance tests (rotation, lighting)
- Performance tests (speed, memory)
- Edge case handling

NLP (Helpdesk Agent):

- Text preprocessing validation
- Intent classification accuracy
- Out-of-vocabulary handling

Recommender System:

- Prediction consistency
- Cold start behavior
- Fairness across user segments

Invariance Testing Example

Ensuring Consistent Behavior:

```
def test_rotation_invariance():  
    image = load_test_image()  
    rotated_image = rotate_image(image, 45)  
  
    original_pred = model.predict(image)  
    rotated_pred = model.predict(rotated_image)  
  
    # Predictions should be similar  
    assert abs(original_pred - rotated_pred) < 0.1
```

VisionaryAI Tests:

- Minor rotations shouldn't change defect detection
- Text typos shouldn't break intent classification
- User preference changes should be gradual

Performance Testing

Production Requirements Testing:

```
def test_inference_speed():  
    model = load_model()  
    batch = create_test_batch(size=100)  
  
    start_time = time.time()  
    predictions = model.predict(batch)  
    end_time = time.time()  
  
    # Must process 100 images in 5 seconds  
    assert (end_time - start_time) < 5.0
```

VisionaryAI's Production Requirements:

- 1000 images/minute processing speed
- <200MB memory usage
- 99.9% uptime

Beyond Accuracy - Comprehensive Evaluation

VisionaryAI's Hard-Learned Lesson:

- 95% accuracy on recommender system
- Terrible business impact
- Problem: Good at obvious predictions, bad at discovery

Comprehensive Framework:

- | | |
|------------------------|---------------------------------|
| 1. Performance Metrics | - Traditional ML metrics |
| 2. Business Metrics | - Stakeholder-relevant measures |
| 3. Fairness Metrics | - Equitable treatment |
| 4. Robustness Metrics | - Stress testing |
| 5. Operational Metrics | - Production performance |

Multi-Dimensional Evaluation

VisionaryAI's Evaluation by Domain:

Domain	Performance	Business	Fairness	Robustness	Operational
Vision	Precision/Recall	False negative rate	N/A	Lighting conditions	Latency
NLP	Classification accuracy	Customer satisfaction	Demographics	Typos/OOV	Memory usage
RecSys	Precision@K	Click-through rate	Product coverage	Cold start	Response time

Validation Strategies

VisionaryAI's Validation Approaches:

1. Hold-out Validation

- Time-based splits for temporal data
- Stratified splits for imbalanced data
- Geographic splits for location-dependent models



2. Cross-Validation

- Essential for smaller datasets
- Stratified K-fold for classification



3. Temporal Validation

- Train on past, validate on future
- Rolling window validation
- Walk-forward validation



4. A/B Testing

- Control vs treatment groups
- Statistical significance testing
- Business impact measurement

Model Comparison Framework

Structured Model Selection:

Model	Accuracy	Precision	Recall	Latency	Memory	Business Impact
-----	-----	-----	-----	-----	-----	-----
ResNet50	0.92	0.91	0.89	50ms	200MB	+15% efficiency
EfficientNet	0.94	0.93	0.91	30ms	150MB	+18% efficiency
MobileNet	0.88	0.86	0.87	10ms	50MB	+12% efficiency

Selection Criteria:

- Statistical significance
- Business impact analysis
- Operational feasibility
- Risk assessment

Bias Detection & Fairness

Why Fairness Matters:

- Ethical obligations
- Legal compliance requirements
- Business reputation
- Long-term sustainability

VisionaryAI's Fairness Considerations:

- NLP Agent: Performance across demographics
- Recommender: Coverage across product categories
- Vision: Consistent detection across conditions

Detection Methods:

- Statistical parity testing
- Equalized odds evaluation
- Demographic parity analysis

Code Demo Preview

What We'll Demonstrate:

1. Code Organization Evolution
 - From messy notebook to structured project
2. Configuration Implementation
 - YAML configs and validation
3. Testing in Practice
 - Unit, integration, and model tests

VisionaryAI's Defect Detection System:

- Real-world example progression
- Practical implementation details

Key Takeaways

Today's Essential Concepts:

1. Code Organization

Modular, reusable, testable structure

2. Configuration-Driven Development

Externalize parameters, enable experimentation

3. Comprehensive Testing

Unit, integration, and model-specific tests

4. Multi-Dimensional Evaluation

Beyond accuracy: business, fairness, robustness

5. Systematic Model Selection

Structured comparison and decision-making

Next Session Preview

Model Packaging & Deployment Concepts:

What's Coming:

- Model serialization strategies
- API design for ML models
- Containerization with Docker
- Deployment patterns and scalability
- Security considerations

VisionaryAI Example:

- Packaging defect detection for edge deployment
- Version management and rollback strategies

Questions & Discussion

- Code organization challenges you've faced?
- Testing strategies that worked/didn't work?
- Configuration management experiences?
- Fairness considerations in your domain?

دانشکار

THANK YOU

FOR YOUR ATTENTION!

